

字节跳动 GPU Scale-up 互联技术白皮书



目录 CONTENT

1. 引言	1
2. GPU 架构和互联方案	2
2.1 GPU 架构分析	2
2.2 GPU 互联方案	5
3. 下一代 Scale-up 互联方案	8
3.1 需求分析	8
3.2 网络方案	10
4. EthLink 网络方案	12
4.1 EthLink 协议栈.....	13
4.2 网络拓扑	18
4.3 网络接口	19

01

引言

随着机器学习和人工智能等领域的持续发展，AI 模型对 GPU 集群数据处理能力的需求也在不断提升。AI 应用需要 GPU 集群处理更大的数据集，训练更深的神经网络和处理更多的并发任务，同时还要减少任务执行时间以及提高系统整体效率。这需要 GPU 集群的 Scale-up 网络规模持续增大，扩展到机架级甚至多机架级。

以太网技术应用在 GPU 集群互联架构具有诸多优势，例如：行业领先的高速链路，大容量交换机，成熟的生态系统等。目前，多个行业组织正在开发用于 AI 集群的 Scale-up 网络技术，这些技术或是对以太网进行扩展，或是将以太网部分组件用作构建模块。

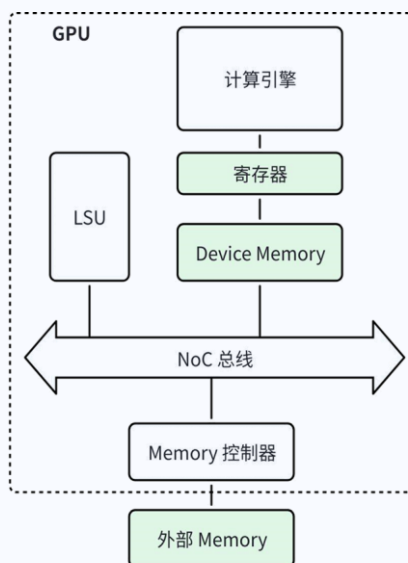
字节跳动基于以太网技术，为 AI 集群提供了低延迟、高带宽的下一代 Scale-up 网络方案，满足了 AI 应用对于 GPU 之间的高速互联传输需求。

02

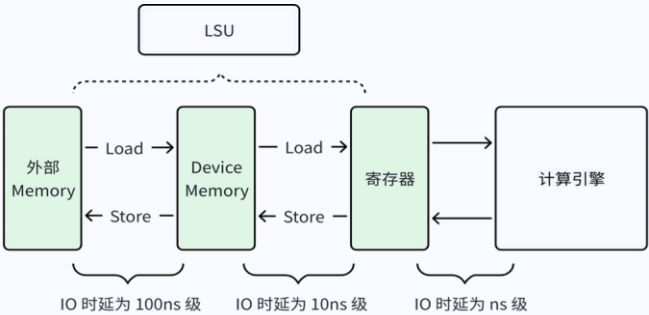
GPU 架构和互联方案

2.1 GPU 架构分析

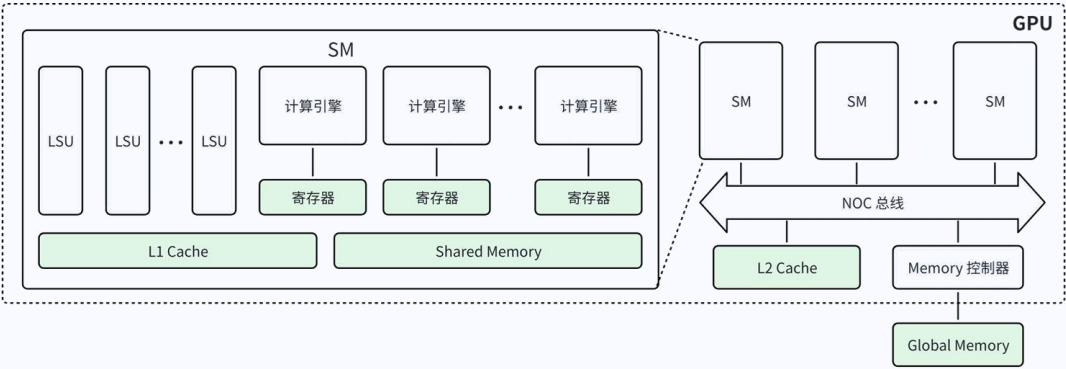
目前主流的 GPU 架构都支持 Load-Store 语义，如下图所示，GPU 的计算引擎从寄存器中读写数据并完成数据的处理，LSU（Load-Store Unit）通过 Load/Store 指令在寄存器和 Device Memory 之间，以及 Device Memory 和外部 Memory 之间完成数据传输。



基于上述架构模型的 GPU，计算引擎主要负责数据的处理，LSU 负责数据的传输，如下图所示，两个模块可以并行工作形成流水线，数据传输主要依靠 Load 和 Store 语义完成。

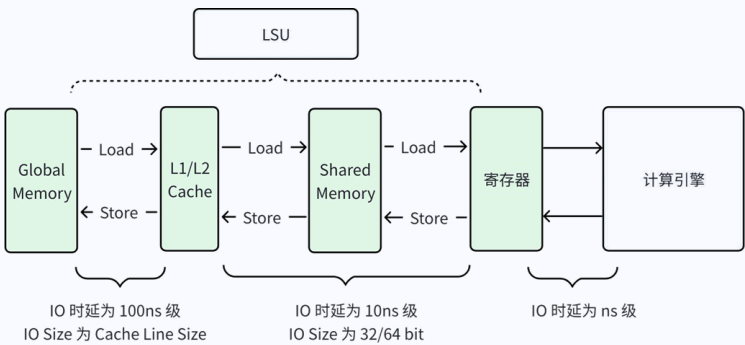


实际的 GPU 架构要比上述的 GPU 架构模型更加复杂，GPGPU（General Purpose GPU）架构通常如下图所示，Device Memory 包括 L1/L2 Cache 和 Shared Memory，Shared Memory 和 L1 Cache 位于 Streaming Multiprocessor（SM）内部，不能在 SM 之间进行共享。L2 Cache 位于 SM 外部，可以被所有 SM 共享。GPU 外部 Memory 为 Global Memory，可以被所有的 SM 访问，也可以被 CPU 或者其他 GPU 访问。



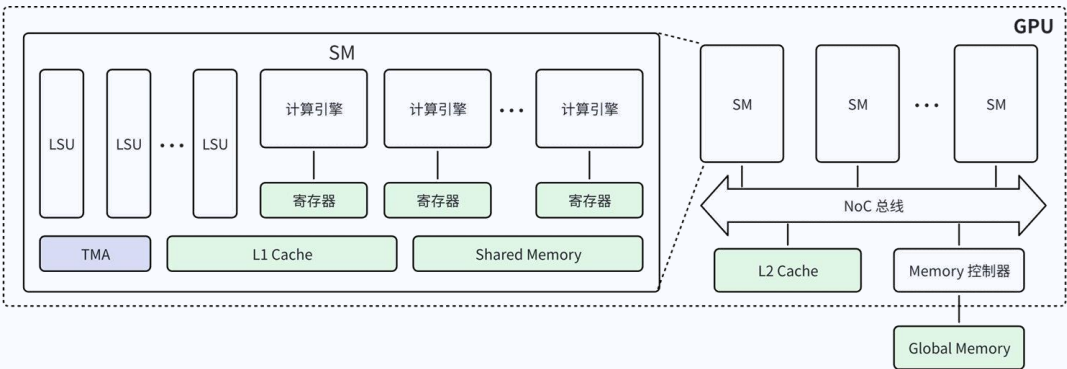
在上述 GPU 的架构中，GPU 主要通过 LSU 完成数据的传输，如下图所示。计算引擎通过寄存器进行数据的访问，IO 时延为 ns 级。LSU 通过 Load/Store 操作实现 Shared Memory 与寄存器之间，以及 Shared Memory 与 L1/L2 Cache 之间的数据传输，IO 时延为 10ns 级，IO Size 通常为寄存器级（32/64 bit）。当出现 Cache Miss 时，LSU 需要进行 Global Memory 和

Shared Memory 之间的数据传输，IO 时延为 100ns 级，IO Size 为 Cache Line Size (64/128/256 Byte) 。

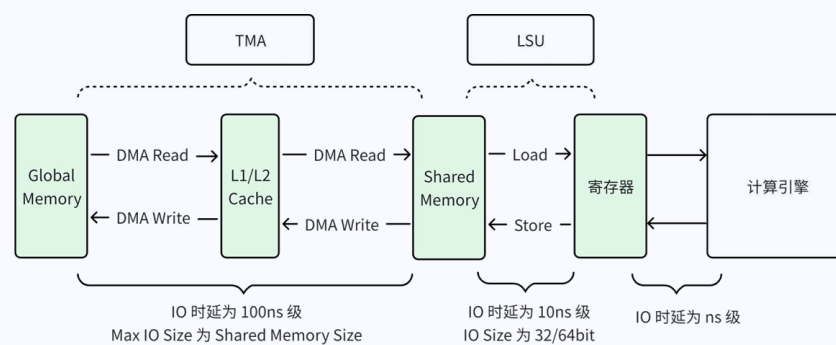


在 AI 应用场景中，计算引擎需要处理大量的数据信息，LSU 可以实现数据的高效传输，但是 LSU 每次传输的数据块比较小，在传输大块数据时，需要 LSU 下发多个 Load/Store 指令来完成数据的搬运。Load/Store 指令的内存地址或者寄存器地址信息，需要计算引擎提前生成并发给 LSU，Load/Store 指令的地址信息处理会消耗计算引擎的算力资源。因此通过 LSU 来完成大块数据的传输，会伴随计算引擎的部分算力资源的消耗。

为了优化 GPU 数据传输方案，降低计算引擎用于数据传输的算力资源，新型号的 GPU 在片内增加了类似于 DMA 引擎的传输模块，如 NVIDIA 从 Hopper 系列的 GPU 开始，在 SM 内增加了 Tensor Memory Accelerator (TMA)，专门用于 Global Memory 到 Shared Memory 之间的数据传输，如下图所示。

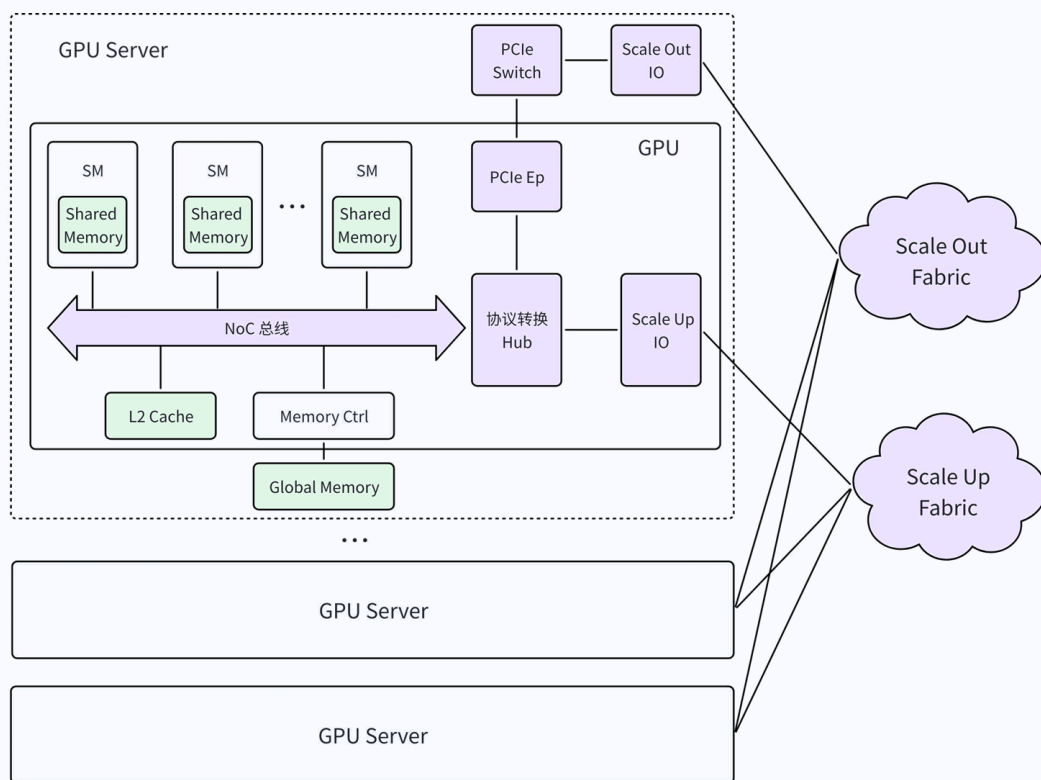


GPU 在增加了 TMA 模块之后，不再需要计算引擎消耗算力资源来在数据传输过程中持续计算 Load/Store 指令的地址信息，只需要计算引擎将数据传输的描述符下发给 TMA 模块，TMA 模块自行计算数据的内存地址信息，独立完成数据在 Global Memory 和 Shared Memory 之间的传输。数据传输流程如下图所示，TMA 通过 DMA Read 和 DMA Write 语义完成 Global Memory 和 Shared Memory 之间的数据传输，LSU 通过 Load/Store 语义完成 Shared Memory 到寄存器之间的数据传输。

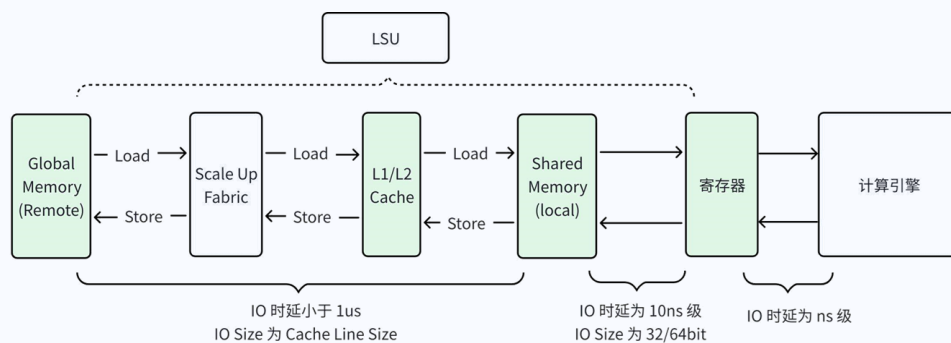


2.2 GPU 互联方案

AI 集群的训练和推理任务，通常需要多个 GPU 协同完成，计算引擎需要处理的数据可能位于多个 GPU 的 Global Memory 中，这时就需要通过 Scale-up 网络和 Scale Out 网络来完成 GPU 之间的数据传输。GPU 的互联架构通常如下图所示，Scale-up 网络的特点是带宽高，时延小，通常基于 Load/Store 语义执行同步操作，业界典型的 Scale-up 协议包括 PCIe，NVLINK，UALINK 等。字节跳动也提出了自研的 Scale-up 网络协议 EthLink，支持 Load/Store 语义和 RDMA 语义，覆盖了 Scale-up 网络的所有应用场景。Scale Out 网络的带宽相对较低，时延相对较高，通常基于 RDMA 语义执行异步操作，业界典型的 Scale Out 协议包括 IB、RoCEv2、UEC 等。

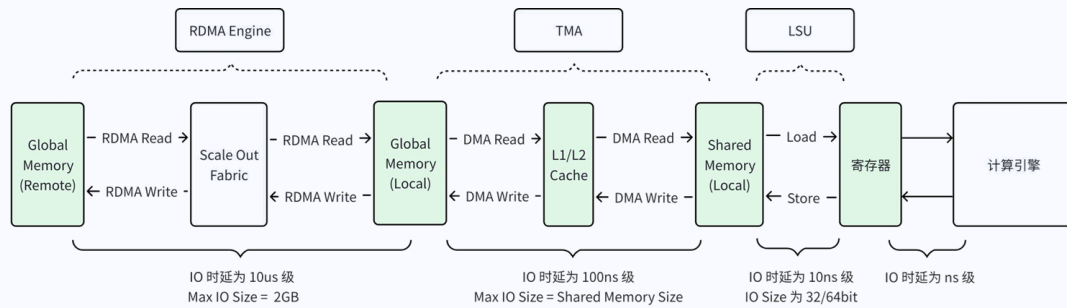


GPU 通过 Scale-up 网络访问远端 GPU Global Memory 的数据流程如下图所示，通过 LSU 实现远端 Global Memory 和本地 Shared Memory 之间的数据传输，数据传输的 IO 时延通常小于 1us，IO Size 通常为 Cache Line Size (64/128/256 Byte)，不同 GPU 之间的内存可通过 Scale-up 网络保证 Cache Coherency。目前业界主流的 GPU 主要通过 Load/Store 操作在 Scale-up 网络实现大块数据的传输，进行数据传输时仍然需要消耗算力资源处理内存地址信息。



GPU 通过 Scale Out 网络访问远端 GPU 的 Global Memory 的数据流程如下图所示，通过 RDMA Engine 实现远端 Global Memory 和本地 Global

Memory 之间的数据传输，数据传输的 IO 时延通常为 10us 级，Max IO Size 为 2GB。GPU 本地 Global Memory 和本地 Shared Memory 之间的数据传输，仍然通过 TMA/LSU 进行处理。



03

下一代 Scale-Up 互联方案

3.1 需求分析

随着 AI 集群规模的增大，Scale-up 网络规模也需要随之增大，Scale-up 网络需要传输的数据量也在逐步增大。通过对当前 GPU 架构和互联方案的分析，可以得到下一代 Scale-up 网络的互联需求如下：

- 1) Load/Store 语义传输小块数据的效率高，时延小，特别适合用于传输控制信息和内存中位置不连续的数据，以及用于对时延要求较高的 AI 推理场景。为了更好的支持推理应用场景，实现小块数据和 GPU 内存中位置不连续的数据的高效传输，Scale-up 网络需要承接 Load/Store 语义。
- 2) Load/Store 语义在数据传输过程中需要计算引擎持续生成 Load/Store 的地址信息，会消耗计算引擎的算力资源。而 RDMA 语义传输大块数据的效率

较高，在数据传输过程中几乎不需要消耗计算引擎的算力。AI 大模型训练中，张量并行、专家并行等模型并行的数据流量需要通过 Scale-up 网络进行传输，模型并行的数据量非常大，通常在 MB 级 ~ GB 级。而且，AI 大模型训练通常会通过优化算法实现数据计算和数据传输的交叠，降低对数据传输时延的要求。通过 RDMA 语义实现大块数据的传输，可以有效节省计算引擎的算力资源。因此，Scale-up 网络需要承接 RDMA 语义。RDMA 语义的缺点也比较明显，交互操作复杂，传输时延大，不适合传输小块数据。

3) Shared Memory 作为 Device Memory，正在发挥越来越重要的作用，Scale-up 网络提供的 RDMA 引擎，既要实现远端 Global Memory 和本地 Global Memory 之间的数据传输，也需要实现远端 Global Memory 和本地 Shared Memory 之间的数据传输。

4) Scale-up 网络的 RDMA 语义，不能完全照搬传统 RDMA 协议，因为传统 RDMA 协议提供的是一套软硬件交互接口，交互操作复杂而且效率低下（相对于纯硬件接口）。Scale-up 网络的 RDMA Engine 是 GPU 内部模块，需要更加简洁的接口来实现 RDMA Engine 与 GPU 内部其他模块之间的交互操作。

5) 随着 Scale-up 网络范围的扩大，通过网络硬件实现 Cache Coherency 的代价越来越大。由于 AI 集群的通信存在明显的周期规律，GPU 之间没有必要继续通过网络硬件来保证 Cache Coherency，可以由系统软件保证 Cache Coherency。

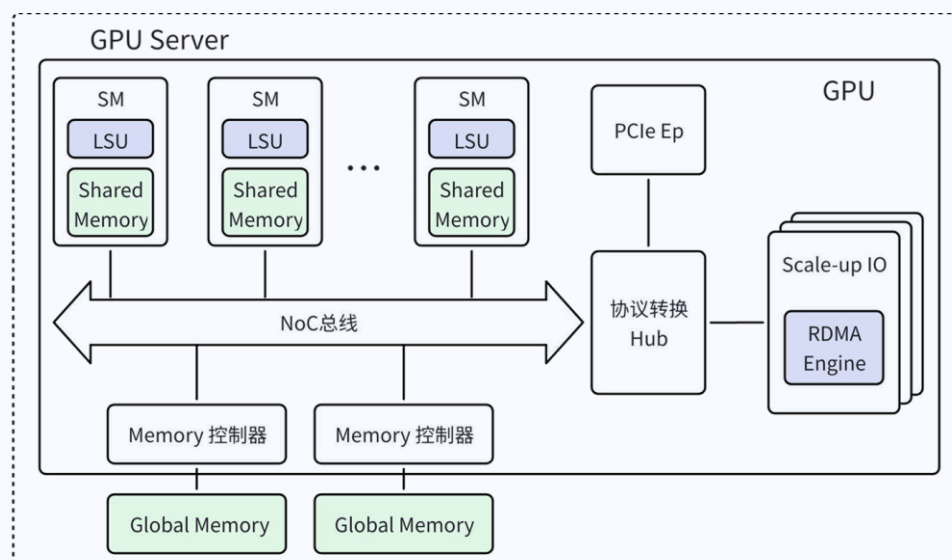
6) Scale-up 网络需要对任意两个 GPU 之间相同传输路径的语义操作和数据报文进行保序，而不同路径上的操作可以乱序执行。

7) 由于单个协议栈的数据处理带宽有限，GPU 通常会部署多个 Scale-up 协议栈实现超大网络带宽。在这种多协议栈的网络架构中，需要保证网络负载在

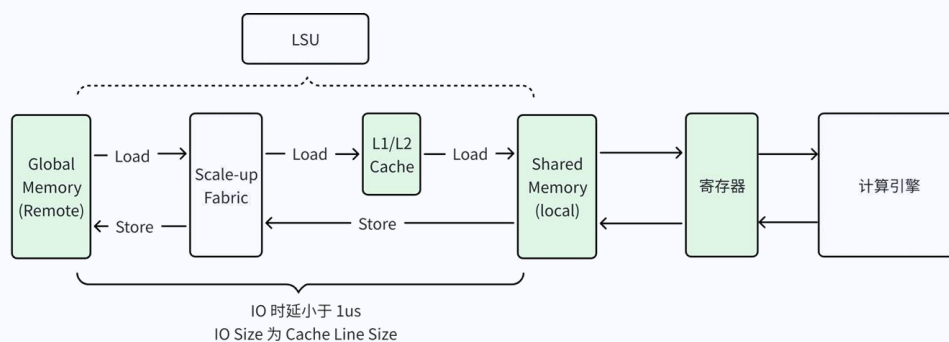
多个协议栈之间，以及在同一个协议栈的多个端口之间尽可能均衡。网络负载均衡引入的报文乱序，需要由 Scale-up 网络的上层应用进行乱序处理。

3.2 网络方案

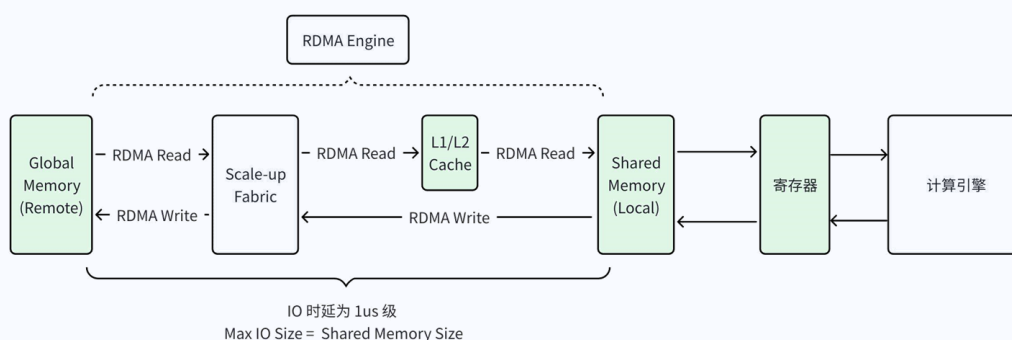
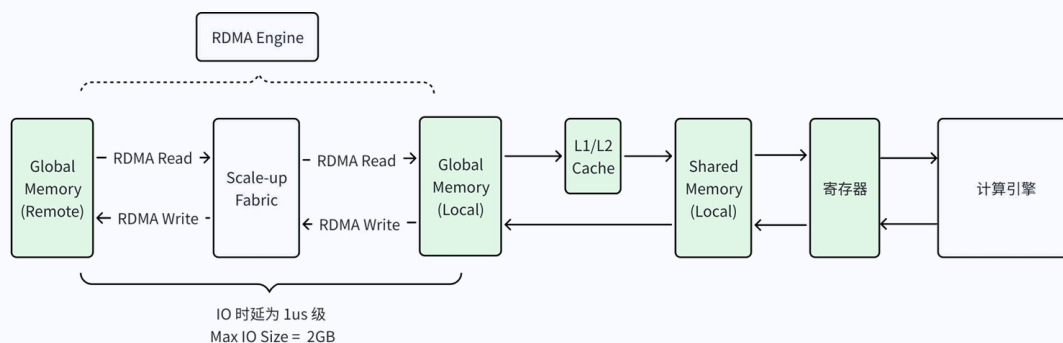
下一代 Scale-up 网络的系统架构如下图所示，GPU 可以通过 Load/Store 语义执行同步操作，Load/Store 语义由 LSU 发起，主要用于数据传输时延敏感的应用场景。GPU 也可以通过 RDMA 语义执行异步操作，RDMA 语义由 RDMA Engine 发起，主要用于带宽大，时延不敏感的应用场景。



Scale-up 网络承接 Load/Store 语义的数据传输流程如下图所示，由于网络硬件不再需要保证 Cache Coherency，L1/L2 Cache 只用于缓存数据，并由系统软件周期性对 Cache 进行 Clear。



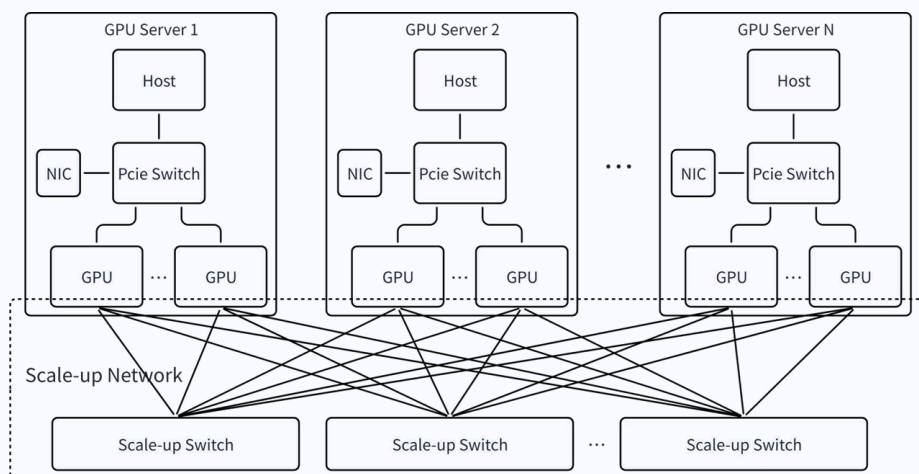
Scale-up 网络承接 RDMA 语义的数据传输流程如下图所示，RDMA Engine 根据描述符的指令类型，完成远端的 Global memory 和本地的 Global Memory 之间，以及远端的 Global Memory 和本地的 Shared Memory 之间的数据传输。



04

EthLink 网络方案

EthLink (Ethernet Link) 是字节跳动自研的 Scale-up 网络协议，为 GPU 集群提供高速互连网络通道，同时承载 GPU 发起的 Load/Store 语义和 RDMA 语义。如下图所示，EthLink 网络范围覆盖了 GPU 服务器内部互联和跨机的 GPU 互联。



Scale-up 最核心的需求是网络带宽：

- PCIe 的优势是天然支持内存语义、低延迟，但它是围绕 CPU 构建的互联，迭代缓慢，带宽落后以太网 2-3 代。
- 以太网的摩尔定律一直有效，18 个月单芯片带宽翻倍。
- 以太网生态成熟开放，大规模部署，低成本。

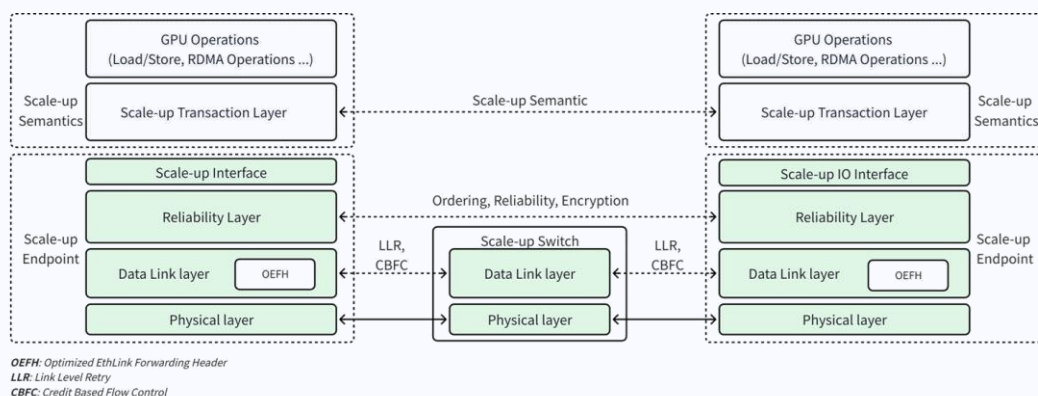
业界各个标准都选择了以太网：

- UALink 的最新标准，物理层已经由 PCIe 切换为以太网。
- ETH-X 也是基于以太网构建。

所以 EthLink 基于以太网构建 Scale-up 互联，分享以太网的生态、成本优势，同时面向 Scale-up 场景，优化标准以太网来支持 Load/Store 语义、RDMA 语义、低延迟、报文载荷效率提升、链路可靠传输等。

4.1 EthLink 协议栈

EthLink 协议栈的系统架构如下图所示，GPU 侧的协议栈可以整体分为两部分，分别是 Scale-Up 语义层和 Scale-up 网络层。Scale-up 语义层可以进一步分为上层 GPU 操作和 Scale-up 事务层。GPU 操作是由 GPU 发起的 Scale-up 网络中的 IO 操作，支持 Load/Store，RDMA 等语义；Scale-up 事务层把 GPU 的操作进一步转换成 Memory Read, Memory Write 等在该层定义的基础操作类型（类似 PCIe 的 memory read, memory write 等操作）。EthLink 的上层应用可以根据实际的应用场景灵活的选择传输语义。Scale-up 网络采用 LLR (Link Layer Retry)和 CBFC (Credit-Based Flow Control)实现可靠的无损网络，高效重传 FEC 无法纠正的错误报文。最后，EthLink 优化了链路层报文头，降低报文头长度，减少传输开销。



4.1.1 GPU 操作

GPU 通过 scale-up 网络能够发起的操作包括 Load/Store 和 RDMA 等，如下表所示。Load/Store 操作通过 LSU 发起，RDMA 操作由 SM 发起并 RDMA Engine 执行。

GPU Operation 类型	描述
Load	从 Global Memory 或者 Shared Memory 加载数据
Store	向 Global Memory 或者 Shared Memory 存储数据
RDMA Write	向远端的 Global Memory 写入数据
RDMA Write with Immediate	向远端的 Global Memory 写入数据，同时把立即数发给远端的 GPU
RDMA Read	从远端的 Global Memory 读回数据
Atomic	原子操作
Message	向远端的 GPU 发送消息

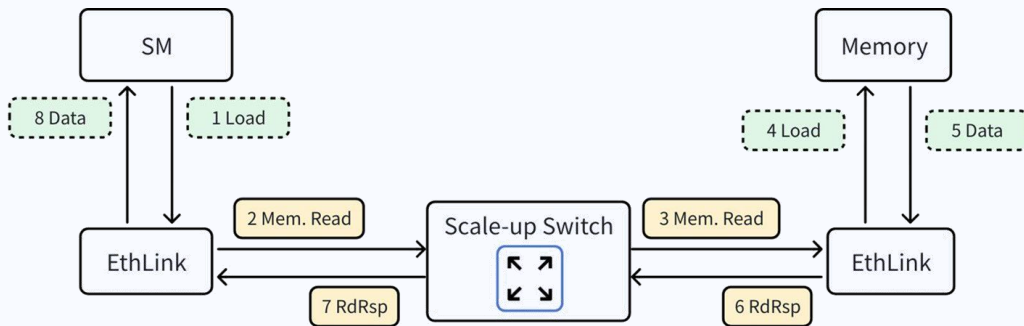
4.1.2 Scale-up 事务层

EthLink 的 Scale-up 事务层定义了 Memory read, Memory write, Atomic 和 Message 语义。Scale-up 事务层接受来自 GPU 的传输操作，转化为 memory read 和 memory write 等语义；最终形成对应的网络报文，通过 Scale-up 网络传输给远端的 GPU。 Scale-up 事务层支持的语义如下表所示：

Transaction 类型	响应类型	描述
Memory Read	Read Response	Memory Read Request, 用来完成内存读操作
Memory Write	Write Response	Memory Write Request, 用来完成内存写操作
Write Message	/	写消息, 用来传递控制信息
Atomic	Write/Read Response	原子操作

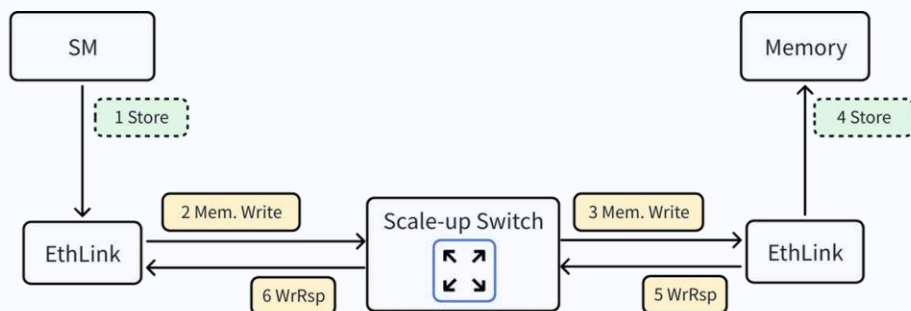
4.1.2.1 Load

Load 操作会被转换为单次的 Memory Read 操作，数据流程如下图所示，SM 发起 Load 操作，会在 EthLink 上生成 Memory Read 报文，实现对远端 GPU Memory 数据的读取。



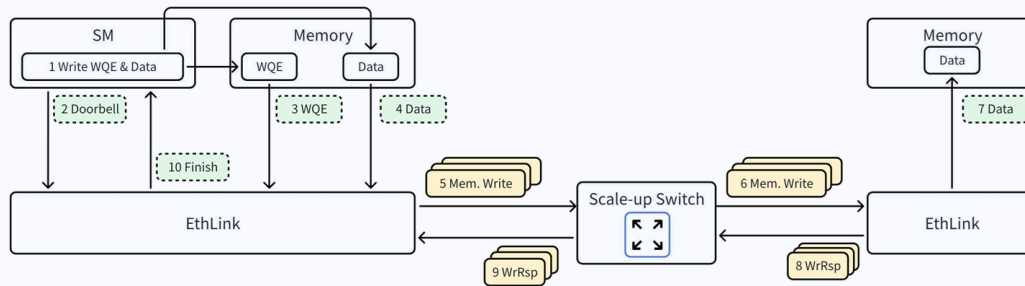
4.1.2.2 Store

Store 操作会被转换为单次的 Memory Write 操作。数据流程如下图所示，SM 发起 Store 操作，并在 EthLink 上生成 Memory Write 报文，把数据通过 Memory Write 操作写入远端 GPU Memory。



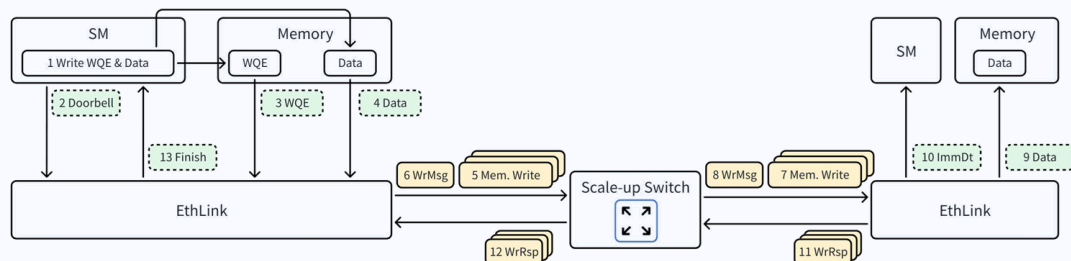
4.1.2.3 RDMA Write

RDMA Write 操作会被转换为若干个 Memory Write 操作。数据流程如下图所示，SM 提前在 Memory 准备好 WQE 和 Data，再下发 Doorbell，EthLink 在收到 Doorbell 通知后，执行 WQE，把 Data 通过 Memory Write 操作写入对端 GPU 的 Memory。



4.1.2.4 RDMA Write with Immediate

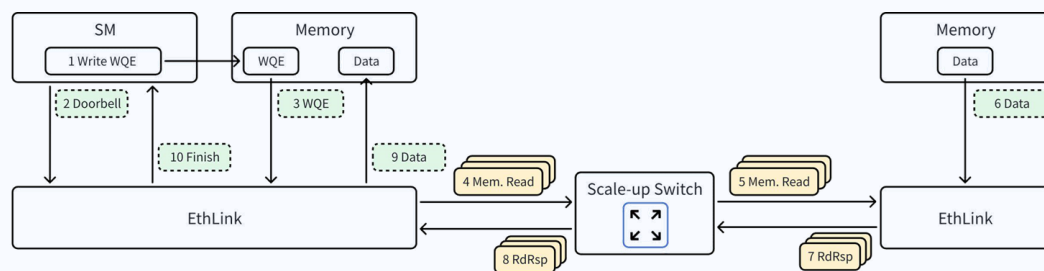
RDMA Write With Immediate 操作会被转换为若干个 Memory Write 操作和 1 个 Write Message 操作。数据流程如下图所示，SM 提前在 Memory 准备好 WQE 和 Data，再下发 Doorbell，EthLink 在收到 Doorbell 通知后，执行 WQE，把 Data 通过 Memory Write 操作写入对端 GPU 的 Memory，并通过 Write Message 操作把 Immediate Data 发给对端 GPU 的 SM。



4.1.2.5 RDMA Read

RDMA Read 操作会被转换为若干个 Memory Read 操作。数据流程如下图所示，SM 提前在 Memory 准备好 WQE，再下发 Doorbell，EthLink 在收到

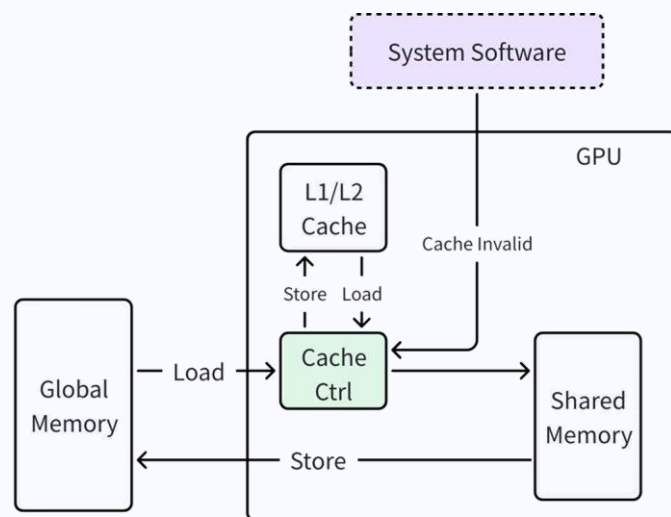
Doorbell 通知后，执行 WQE，通过 Memory Read 操作从对端 GPU Memory 中读回 Data，并写入请求端 GPU 的 Memory。



4.1.3 Cache Coherency

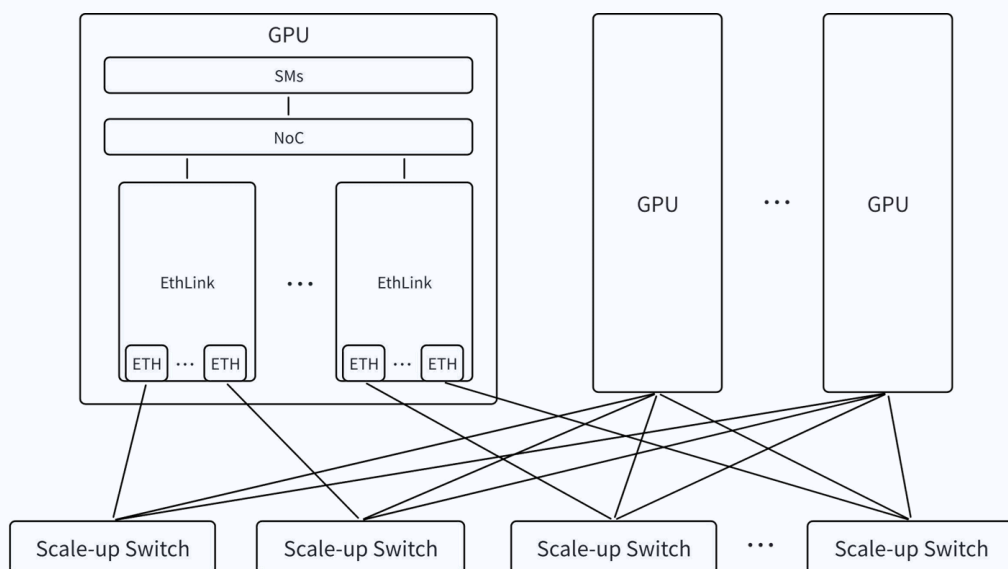
EthLink Cache Coherency 的方案如下图所示，具体工作流程如下：

- 1) 当数据从片外 Global Memory Load 到 GPU 内的 Shared Memory 时，仍然缓存在 Cache 中，以加速 GPU 下一次对相同 Global Memory 地址的数据访问。
- 2) 当数据需要从 GPU 内的 Shared Memory Store 到片外的 Global Memory 中，数据不会写入 Cache，而是直接写入片外的 Global Memory。
- 3) 系统软件在必要的时候清除 Cache，以保证 Cache 缓存的数据和片外 Global Memory 的数据一致性。



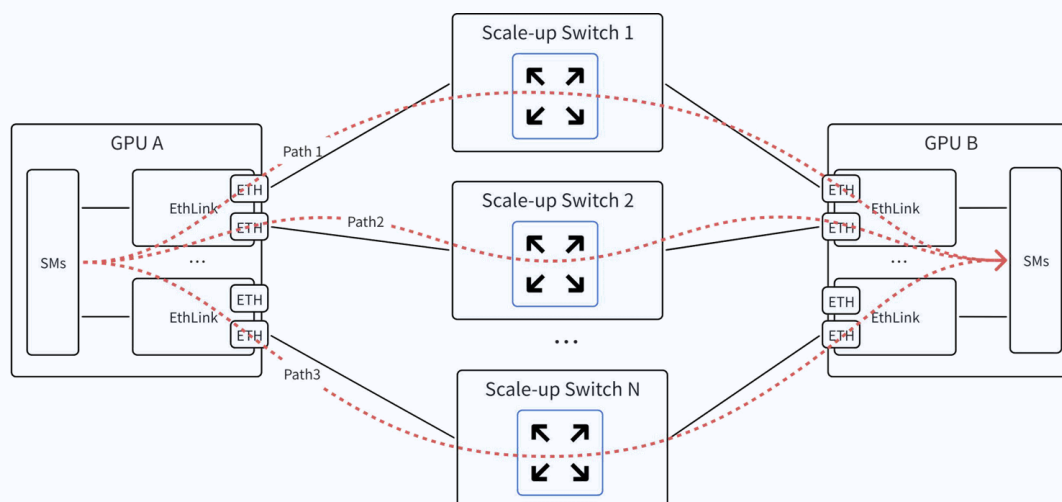
4.2 网络拓扑

EthLink 的网络拓扑如下图所示，每个 GPU 服务器部署多个 EthLink 协议栈，每个 EthLink 协议栈可以支持 1~4 个以太网接口，GPU 服务器之间通过低时延以太网交换机互联，同一个 Scale-up 网络域最大支持 1024 个 GPU 节点。



4.2.1 端口负载均衡

GPU 服务器通常会在部署多个 EthLink 协议栈来大幅度提升 GPU 服务器的 Scale-up 网络带宽。当两个 GPU 之间有数据传输需求时，如下图所示，GPU 之间可以通过使用 Multi-Path 来提升实现负载均衡，从而进一步提升网络带宽的利用率。

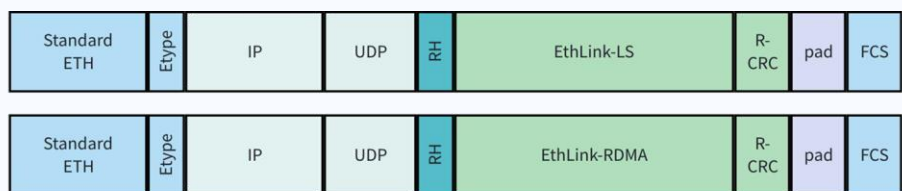


EthLink 使用通过 Multi-Path 实现负载均衡后，有可能会引入乱序。由于乱序涉及到多个协议栈的报文，无法在 EthLink 上实现保序，需要通过 EthLink 的上层应用处理乱序问题。

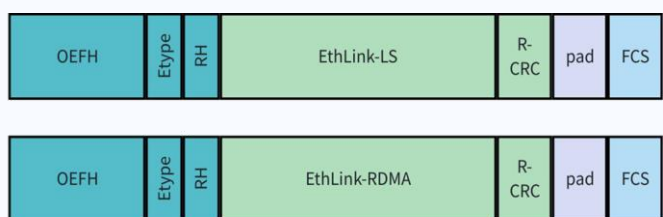
4.3 网络接口

4.3.1 报文封装

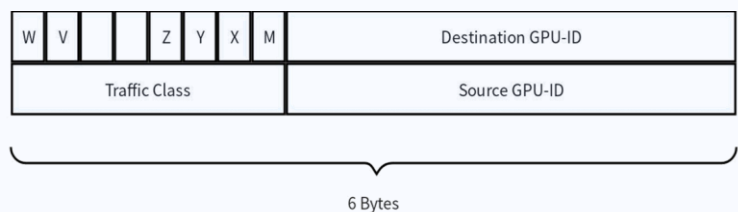
如上所述，EthLink 的事务层语义可以承载 GPU 的 Load/Store 和/或 RDMA 操作。本小节中为了体现上层 GPU 操作的不同，分别用 EthLink-LS 和 EthLink-RDMA 来体现载荷中的具体内容。EthLink 的报文格式如下图所示，其中 RH（Reliability Header）用于保证端到端可靠性，是对 LLR 提供的链路层可靠性的增强。



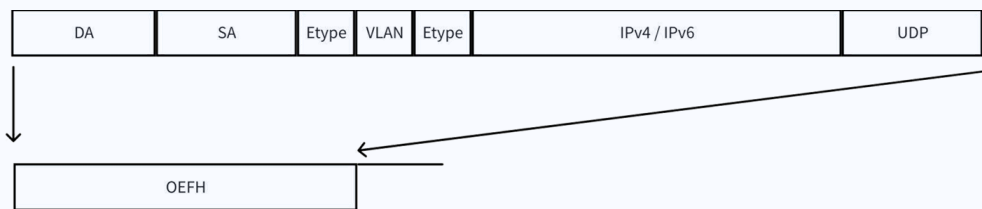
同时，为了解决传统 IP 报文有效载荷率低的问题，EthLink 也设计了优化的报文头部，即 OEFH（Optimized EthLink Forwarding Header），如下图所示。OEFH 使用更小的报文 header 来提升报文有效负载率。同时与现有 L2 层报文头长度对齐，能够兼容现有的以太网报文。



OEFH 的格式如下图所示，包含 source 和 destination GPU ID，交换机根据 GPU ID 来转发报文。



OEFH 能够取代标准以太网和 TCP/IP 协议栈中的 ETH+IP+UDP 报文头，显著缩短了报文头长度，降低了报文开销。



4.3.2 FEC

Scale-up 网络对延迟敏感，而标准 FEC 引入的延时占链路层延迟的很大比重。因此可以选择 RS-272 这种更低延迟的 FEC。

4.3.3 链路层可靠传输

当 Scale-up 从机内短距离点对点互联拓展到机外，甚至跨机柜互联，且经过交换机转发时，网络的丢包概率显著升高，而检测丢包、丢包重传给协议层带来了许多复杂度，同时端到端的重传又带来了更多的延迟。

所以 EthLink 支持链路层的可靠传输。丢包主要分 2 个原因，一是链路上的丢包，比如 CRC 错误，这一类由 LLR 来解决，另一种是交换机内部丢包，由 CBFC 来解决。

4.3.3.1 Link Level Retry (LLR)

链路的发送端先缓存报文，如果发生报文丢包，则发起重传，直到对端确认收到该报文。链路层的重传，可以降低 FEC 的要求，使选择 RS-272 这种低延迟 FEC 成为可能。另外，也降低了链路质量的要求，如果存在光互联，则可以引入 LPO，来降低光互联上的延迟。

4.3.3.2 Credit Based Flow Control (CBFC)

交换机内部 buffer 的丢包，通常选择 PFC 机制，也可选择 CBFC。CBFC 提供更细粒度的控制，可与 VC 绑定来使用，同时因为基于 Credit，可以精确的知道对端 Buffer 的情况，提供更有效的机制以实现更低的延迟。

4.3.4 Switch Event Notification

Scale-up 考虑更低延迟，通常采取单层交换机的方式，如下图所示。如果发生链路断开，交换机没有路径的冗余，无法直接切换路径来避免丢包，只能通过 GPU 侧 multi-path 来切换路径，而 Source GPU 无法感知跨交换机的远端链路状态，因此丢包可能会持续发生。

所以交换机需要与 GPU 间建立状态反馈机制，例如在 port down 时需要通过 Switch Event Notification，快速发送 port event 给 Source GPU，来达到快速切换路径。

